The Ultimate DSA Sheet

Foundation drills, a topic-wise master sheet, company filters and a daily tracking system.

40 foundation problems | **232** master-sheet problems | **6** company profiles | spaced-repetition tracker included

Contents

- Part 1 - The Foundation & Prerequisite Core
- Part 2 - Topic-Wise Master Sheet (Easy to Hard)
- Part 3 - High-Yield Company-Wise Filters
- Part 4 - Daily Strategy & Tracking Template
- Appendix - Codeforces / AtCoder / CSES bridge

How to read this sheet

Link column	Every LeetCode entry shows **LC/<slug>** and is clickable: it opens leetcode.com/problems/<slug>/. If a link ever fails, search the problem name.
(Premium)	Needs LeetCode Premium. Free alternatives: NeetCode.io, LintCode, or the same problem on GeeksforGeeks.
[B75]	Problem is on the Blind 75 list. Problems without it come from NeetCode 150/All, Striver's A2Z or Love Babbar's sheet.
Non-LeetCode	Entries like GfG / SPOJ show the platform and a search title instead of a link.
Difficulty	**Easy**, **Medium**, **Hard** as labelled on LeetCode (or the closest equivalent elsewhere).
Done column	Tick the box once you solve the problem *without* reading the solution. The revision schedule is in Part 4.

Suggested pacing: Part 1 in week 1, Part 2 topic by topic in the order shown, start contests from week 3, and run the Part 4 tracker from day one. Finishing every problem is not the goal; finishing each topic's Easy and Medium rows and re-solving the ones you missed is.

Compiled from the public Striver, NeetCode, Blind 75 and Babbar lists. Problem membership across sheets was consolidated by hand; verify a specific problem's difficulty on the platform before relying on it.

PART 1: The Foundation & Prerequisite Core

Warm-up drills that build muscle memory before the heavy data structures. Solve each row in order, then move to Part 2.

Two Pointers / Sliding Window - Two Pointers

Topic	Problem Name / Concept	Why it matters / What it teaches	Difficulty	Done
Two Pointers	**Valid Palindrome** LC/valid-palindrome [B75]	Opposite-end pointers with skip conditions for non-alphanumerics	**Easy**	[ ]
Two Pointers	**Two Sum II - Input Array Is Sorted** LC/two-sum-ii-input-array-is-sorted	Sorted input: move pointers by comparing the sum to target (O(n), O(1) space)	**Medium**	[ ]
Two Pointers	**Move Zeroes** LC/move-zeroes	Reader/writer pointers: in-place stable partition	**Easy**	[ ]
Two Pointers	**Remove Duplicates from Sorted Array** LC/remove-duplicates-from-sorted-array	Slow/fast pointer in-place compaction	**Easy**	[ ]
Two Pointers	**Container With Most Water** LC/container-with-most-water [B75]	Greedy pointer move: always discard the shorter wall	**Medium**	[ ]
Two Pointers	**3Sum** LC/3sum [B75]	Sort + fix one + two pointers; duplicate skipping	**Medium**	[ ]

Two Pointers / Sliding Window - Sliding Window

Topic	Problem Name / Concept	Why it matters / What it teaches	Difficulty	Done
Sliding Window	**Maximum Average Subarray I** LC/maximum-average-subarray-i	Fixed-size window: add right element, drop left element	**Easy**	[ ]
Sliding Window	**Contains Duplicate II** LC/contains-duplicate-ii	Fixed window + hash set of the last k elements	**Easy**	[ ]
Sliding Window	**Longest Substring Without Repeating Characters** LC/longest-substring-without-repeating-characters [B75]	Variable window: expand right, shrink left until valid	**Medium**	[ ]
Sliding Window	**Fruit Into Baskets** LC/fruit-into-baskets	Window with at most K distinct values (template for many problems)	**Medium**	[ ]

Core Recursion - Recursion

Topic	Problem Name / Concept	Why it matters / What it teaches	Difficulty	Done
Recursion	**Print 1 to N (recursion only)** GfG: Print 1 to N without loop	Base case + call order: print before vs after the recursive call	**Easy**	[ ]
Recursion	**Fibonacci Number** LC/fibonacci-number	Overlapping subproblems: the doorway to memoization	**Easy**	[ ]
Recursion	**Reverse String (recursive variant)** LC/reverse-string	Recursion on a shrinking range	**Easy**	[ ]
Recursion	**Pow(x, n)** LC/powx-n	Divide and conquer: O(log n) exponentiation; handle negative n	**Medium**	[ ]
Recursion	**Sort an Array (merge sort)** LC/sort-an-array	Divide, recurse, merge; read the recursion tree for O(n log n)	**Medium**	[ ]
Recursion	**Subsets** LC/subsets	Include/exclude decision tree: the core template of all backtracking	**Medium**	[ ]
Recursion	**Subsets II** LC/subsets-ii	Sort + skip duplicates at the same recursion level	**Medium**	[ ]
Recursion	**Generate Parentheses** LC/generate-parentheses	Pruned choices: open < n, close < open	**Medium**	[ ]
Recursion	**Letter Combinations of a Phone Number** LC/letter-combinations-of-a-phone-number	Multi-branch recursion indexed by digit position	**Medium**	[ ]

Bit Manipulation

Topic	Problem Name / Concept	Why it matters / What it teaches	Difficulty	Done
Bit Manipulation	**Number of 1 Bits** LC/number-of-1-bits [B75]	n & (n-1) clears the lowest set bit	**Easy**	[ ]
Bit Manipulation	**Power of Two** LC/power-of-two	A power of two has exactly one set bit	**Easy**	[ ]
Bit Manipulation	**Counting Bits** LC/counting-bits [B75]	dp[i] = dp[i >> 1] + (i & 1)	**Easy**	[ ]
Bit Manipulation	**Reverse Bits** LC/reverse-bits [B75]	Shift-and-mask loop over 32 bits	**Easy**	[ ]
Bit Manipulation	**Single Number** LC/single-number	XOR cancels pairs	**Easy**	[ ]
Bit Manipulation	**Missing Number** LC/missing-number [B75]	XOR of indices and values (or sum formula)	**Easy**	[ ]
Bit Manipulation	**Sum of Two Integers** LC/sum-of-two-integers [B75]	Add without +: XOR gives the sum, (a & b) << 1 gives the carry	**Medium**	[ ]
Bit Manipulation	**Single Number II** LC/single-number-ii	Per-bit counting modulo 3	**Medium**	[ ]
Bit Manipulation	**Bitwise AND of Numbers Range** LC/bitwise-and-of-numbers-range	Common binary prefix of the range endpoints	**Medium**	[ ]
Bit Manipulation	**Divide Two Integers** LC/divide-two-integers	Shift-and-subtract division without / or *	**Medium**	[ ]
Bit Manipulation	**Subsets (bitmask solution)** LC/subsets	Enumerate masks 0..2^n - 1: the bridge to bitmask DP	**Medium**	[ ]

Matrix Manipulation - Matrix

Topic	Problem Name / Concept	Why it matters / What it teaches	Difficulty	Done
Matrix	**Transpose Matrix** LC/transpose-matrix	Index mapping (i, j) -> (j, i)	**Easy**	[ ]
Matrix	**Pascal's Triangle** LC/pascals-triangle	Build each row from the previous one	**Easy**	[ ]
Matrix	**Flood Fill** LC/flood-fill	Grid DFS/BFS with direction arrays: template for every grid problem	**Easy**	[ ]
Matrix	**Rotate Image** LC/rotate-image [B75]	Transpose, then reverse each row (in place)	**Medium**	[ ]
Matrix	**Spiral Matrix** LC/spiral-matrix [B75]	Boundary shrinking with four pointers	**Medium**	[ ]
Matrix	**Set Matrix Zeroes** LC/set-matrix-zeroes [B75]	O(1) space: use first row/column as markers	**Medium**	[ ]
Matrix	**Search a 2D Matrix II** LC/search-a-2d-matrix-ii	Staircase search from the top-right corner	**Medium**	[ ]
Matrix	**Valid Sudoku** LC/valid-sudoku	Hash sets per row/column/box; box index = (r//3)*3 + c//3	**Medium**	[ ]
Matrix	**Diagonal Traverse** LC/diagonal-traverse	Direction toggling with boundary handling	**Medium**	[ ]
Matrix	**Game of Life** LC/game-of-life	In-place state encoding (old|new in one cell)	**Medium**	[ ]

PART 2: Topic-Wise Master Sheet (Easy to Hard)

Topics follow a learning order (Heaps come before Graphs because Dijkstra needs a priority queue). Inside each table, difficulty rises from top to bottom.

Arrays & Hashing

Learn first: Prefix sums, hash map/set, in-place partitioning.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
ARR-01	Two Sum	LC/two-sum [B75]	Easy	Hash map complement lookup	[ ]
ARR-02	Contains Duplicate	LC/contains-duplicate [B75]	Easy	Hash set membership	[ ]
ARR-03	Valid Anagram	LC/valid-anagram [B75]	Easy	Frequency count (26-slot array)	[ ]
ARR-04	Best Time to Buy and Sell Stock	LC/best-time-to-buy-and-sell-stock [B75]	Easy	Running minimum, single pass	[ ]
ARR-05	Majority Element	LC/majority-element	Easy	Boyer-Moore voting	[ ]
ARR-06	Merge Sorted Array	LC/merge-sorted-array	Easy	Two pointers filling from the back	[ ]
ARR-07	Group Anagrams	LC/group-anagrams [B75]	Medium	Hash by canonical key (sorted string / count tuple)	[ ]
ARR-08	Top K Frequent Elements	LC/top-k-frequent-elements [B75]	Medium	Bucket sort by frequency / heap	[ ]
ARR-09	Product of Array Except Self	LC/product-of-array-except-self [B75]	Medium	Prefix and suffix products	[ ]
ARR-10	Maximum Subarray	LC/maximum-subarray [B75]	Medium	Kadane's algorithm	[ ]
ARR-11	Maximum Product Subarray	LC/maximum-product-subarray [B75]	Medium	Track running max and min (sign flips)	[ ]
ARR-12	Subarray Sum Equals K	LC/subarray-sum-equals-k	Medium	Prefix sum + hash map of counts	[ ]
ARR-13	Longest Consecutive Sequence	LC/longest-consecutive-sequence [B75]	Medium	Hash set; only start counting at sequence heads	[ ]
ARR-14	Sort Colors	LC/sort-colors	Medium	Dutch national flag (3-way partition)	[ ]
ARR-15	Next Permutation	LC/next-permutation	Medium	Find pivot, swap with successor, reverse suffix	[ ]
ARR-16	Majority Element II	LC/majority-element-ii	Medium	Extended Boyer-Moore (n/3 candidates)	[ ]
ARR-17	Find the Duplicate Number	LC/find-the-duplicate-number	Medium	Floyd's cycle detection on the index graph	[ ]
ARR-18	First Missing Positive	LC/first-missing-positive	Hard	Cyclic sort / use the array as a hash	[ ]
ARR-19	Trapping Rain Water	LC/trapping-rain-water	Hard	Two pointers with running left/right max	[ ]
ARR-20	Reverse Pairs	LC/reverse-pairs	Hard	Merge sort with counting step	[ ]

Strings (Advanced Windows, Palindromes, KMP)

Learn first: Frequency windows, expand-around-center, prefix function (LPS).

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
STR-01	Longest Common Prefix	LC/longest-common-prefix	Easy	Vertical scan	[ ]
STR-02	Find the Index of the First Occurrence in a String	LC/find-the-index-of-the-first-occurrence-in-a-string	Easy	KMP / Z-function	[ ]
STR-03	Reverse Words in a String	LC/reverse-words-in-a-string	Medium	Split/trim; in-place two-pointer variant	[ ]
STR-04	String to Integer (atoi)	LC/string-to-integer-atoi	Medium	State handling + overflow clamping	[ ]
STR-05	Longest Palindromic Substring	LC/longest-palindromic-substring [B75]	Medium	Expand around center (O(n^2))	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
STR-06	Palindromic Substrings	LC/palindromic-substrings [B75]	**Medium**	Expand around center, count all	[ ]
STR-07	Longest Repeating Character Replacement	LC/longest-repeating-character-replacement [B75]	**Medium**	Window invariant: length - maxFreq <= k	[ ]
STR-08	Permutation in String	LC/permutation-in-string	**Medium**	Fixed window, frequency-array match	[ ]
STR-09	Minimum Remove to Make Valid Parentheses	LC/minimum-remove-to-make-valid-parentheses	**Medium**	Stack of indices	[ ]
STR-10	Repeated DNA Sequences	LC/repeated-dna-sequences	**Medium**	Rolling hash / hash set of fixed windows	[ ]
STR-11	Encode and Decode Strings	LC/encode-and-decode-strings (Premium) [B75]	**Medium**	Length-prefix serialization (Premium; also on NeetCode/LintCode)	[ ]
STR-12	Minimum Window Substring	LC/minimum-window-substring [B75]	**Hard**	Variable window with need/have counters	[ ]
STR-13	Shortest Palindrome	LC/shortest-palindrome	**Hard**	KMP prefix function on s + '#' + reverse(s)	[ ]
STR-14	Longest Happy Prefix	LC/longest-happy-prefix	**Hard**	Prefix function (LPS array)	[ ]

Binary Search

Learn first: Lower/upper bound templates; searching on the answer space with a feasibility check.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
BS-01	Binary Search	LC/binary-search	**Easy**	Classic template	[ ]
BS-02	Search Insert Position	LC/search-insert-position	**Easy**	Lower bound	[ ]
BS-03	Sqrt(x)	LC/sqrtx	**Easy**	Binary search on answer	[ ]
BS-04	Find First and Last Position of Element in Sorted Array	LC/find-first-and-last-position-of-element-in-sorted-array	**Medium**	Lower bound + upper bound	[ ]
BS-05	Search a 2D Matrix	LC/search-a-2d-matrix	**Medium**	Flattened-index binary search	[ ]
BS-06	Search in Rotated Sorted Array	LC/search-in-rotated-sorted-array [B75]	**Medium**	Identify which half is sorted	[ ]
BS-07	Find Minimum in Rotated Sorted Array	LC/find-minimum-in-rotated-sorted-array [B75]	**Medium**	Compare mid with the right end	[ ]
BS-08	Single Element in a Sorted Array	LC/single-element-in-a-sorted-array	**Medium**	Pair-parity binary search	[ ]
BS-09	Find Peak Element	LC/find-peak-element	**Medium**	Slope-based binary search	[ ]
BS-10	Koko Eating Bananas	LC/koko-eating-bananas	**Medium**	Binary search on answer + feasibility check	[ ]
BS-11	Capacity To Ship Packages Within D Days	LC/capacity-to-ship-packages-within-d-days	**Medium**	Binary search on answer (minimize the maximum)	[ ]
BS-12	Time Based Key-Value Store	LC/time-based-key-value-store	**Medium**	Binary search over sorted timestamps	[ ]
BS-13	Split Array Largest Sum	LC/split-array-largest-sum	**Hard**	Minimize the maximum: BS on answer + greedy check	[ ]
BS-14	Median of Two Sorted Arrays	LC/median-of-two-sorted-arrays	**Hard**	Binary search on the partition	[ ]
BS-15	Aggressive Cows	SPOJ AGGRCOW / Coding Ninjas	**Hard**	Maximize the minimum distance (BS on answer); Striver A2Z staple	[ ]

Linked List

Learn first: Dummy head, fast/slow pointers, in-place reversal.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
LL-01	Reverse Linked List	LC/reverse-linked-list [B75]	**Easy**	Iterative pointer reversal	[ ]
LL-02	Merge Two Sorted Lists	LC/merge-two-sorted-lists [B75]	**Easy**	Dummy head + two pointers	[ ]
LL-03	Linked List Cycle	LC/linked-list-cycle [B75]	**Easy**	Floyd's cycle detection	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
LL-04	Middle of the Linked List	LC/middle-of-the-linked-list	Easy	Fast/slow pointers	[ ]
LL-05	Palindrome Linked List	LC/palindrome-linked-list	Easy	Find middle + reverse second half	[ ]
LL-06	Intersection of Two Linked Lists	LC/intersection-of-two-linked-lists	Easy	Pointer switching (length equalization)	[ ]
LL-07	Remove Nth Node From End of List	LC/remove-nth-node-from-end-of-list [B75]	Medium	Gap of n between two pointers	[ ]
LL-08	Linked List Cycle II	LC/linked-list-cycle-ii	Medium	Floyd phase 2: find the cycle start	[ ]
LL-09	Add Two Numbers	LC/add-two-numbers	Medium	Digit-wise addition with carry	[ ]
LL-10	Swap Nodes in Pairs	LC/swap-nodes-in-pairs	Medium	Pointer rewiring with dummy head	[ ]
LL-11	Rotate List	LC/rotate-list	Medium	Make circular, break at (len - k)	[ ]
LL-12	Reorder List	LC/reorder-list [B75]	Medium	Middle + reverse + interleave merge	[ ]
LL-13	Sort List	LC/sort-list	Medium	Merge sort on a linked list	[ ]
LL-14	Copy List with Random Pointer	LC/copy-list-with-random-pointer	Medium	Hash map or interleaved-node trick	[ ]
LL-15	LRU Cache	LC/lru-cache	Medium	Hash map + doubly linked list	[ ]
LL-16	Reverse Nodes in k-Group	LC/reverse-nodes-in-k-group	Hard	Segment-wise reversal with reconnection	[ ]
LL-17	Merge k Sorted Lists	LC/merge-k-sorted-lists [B75]	Hard	Min-heap / divide and conquer merge	[ ]

Stacks, Queues & Monotonic Structures

Learn first: Stack for matching/nesting; monotonic stack for next-greater/smaller; deque for window extremes.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
STK-01	Valid Parentheses	LC/valid-parentheses [B75]	Easy	Stack matching	[ ]
STK-02	Implement Queue using Stacks	LC/implement-queue-using-stacks	Easy	Two stacks, amortized O(1)	[ ]
STK-03	Next Greater Element I	LC/next-greater-element-i	Easy	Monotonic stack + hash map	[ ]
STK-04	Min Stack	LC/min-stack	Medium	Auxiliary stack tracking minimum	[ ]
STK-05	Evaluate Reverse Polish Notation	LC/evaluate-reverse-polish-notation	Medium	Operand stack	[ ]
STK-06	Daily Temperatures	LC/daily-temperatures	Medium	Monotonic decreasing stack	[ ]
STK-07	Next Greater Element II	LC/next-greater-element-ii	Medium	Circular array: double pass	[ ]
STK-08	Decode String	LC/decode-string	Medium	Nested stack (count, partial string)	[ ]
STK-09	Asteroid Collision	LC/asteroid-collision	Medium	Stack simulation	[ ]
STK-10	Car Fleet	LC/car-fleet	Medium	Sort by position, stack of arrival times	[ ]
STK-11	Online Stock Span	LC/online-stock-span	Medium	Monotonic stack storing spans	[ ]
STK-12	Remove K Digits	LC/remove-k-digits	Medium	Greedy monotonic increasing stack	[ ]
STK-13	Sum of Subarray Minimums	LC/sum-of-subarray-minimums	Medium	Monotonic stack: contribution technique (prev/next smaller)	[ ]
STK-14	Largest Rectangle in Histogram	LC/largest-rectangle-in-histogram	Hard	Monotonic stack: prev/next smaller bars	[ ]
STK-15	Maximal Rectangle	LC/maximal-rectangle	Hard	Histogram per row	[ ]
STK-16	Basic Calculator	LC/basic-calculator	Hard	Stack with sign/parenthesis context	[ ]
STK-17	Sliding Window Maximum	LC/sliding-window-maximum	Hard	Monotonic deque	[ ]

Recursion & Backtracking

Learn first: Choose / explore / un-choose; pruning; duplicate handling. (Subsets, Generate Parentheses and Letter Combinations are in Part 1.)

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
BT-01	Combination Sum	LC/combination-sum [B75]	**Medium**	Unbounded choice, non-decreasing index	[ ]
BT-02	Combination Sum II	LC/combination-sum-ii	**Medium**	Sort + skip same-level duplicates	[ ]
BT-03	Permutations	LC/permutations	**Medium**	Used-array or swap-based permutation	[ ]
BT-04	Permutations II	LC/permutations-ii	**Medium**	Sort + duplicate skipping with used[]	[ ]
BT-05	Word Search	LC/word-search [B75]	**Medium**	Grid DFS with visited backtracking	[ ]
BT-06	Palindrome Partitioning	LC/palindrome-partitioning	**Medium**	Partition backtracking + palindrome check	[ ]
BT-07	Restore IP Addresses	LC/restore-ip-addresses	**Medium**	Segment-length constrained backtracking	[ ]
BT-08	Partition to K Equal Sum Subsets	LC/partition-to-k-equal-sum-subsets	**Medium**	Bucket filling with pruning (bridge to bitmask DP)	[ ]
BT-09	N-Queens	LC/n-queens	**Hard**	Column/diagonal sets for O(1) conflict check	[ ]
BT-10	Sudoku Solver	LC/sudoku-solver	**Hard**	Constraint backtracking with row/col/box sets	[ ]
BT-11	Rat in a Maze	GfG: Rat in a Maze Problem I	**Medium**	Grid path enumeration with backtracking (Striver A2Z)	[ ]

Trees & Binary Search Trees

Learn first: DFS return-value patterns, BFS by level, BST invariant (inorder is sorted).

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
TREE-01	Invert Binary Tree	LC/invert-binary-tree [B75]	**Easy**	Recursive swap	[ ]
TREE-02	Maximum Depth of Binary Tree	LC/maximum-depth-of-binary-tree [B75]	**Easy**	DFS height	[ ]
TREE-03	Same Tree	LC/same-tree [B75]	**Easy**	Simultaneous DFS	[ ]
TREE-04	Symmetric Tree	LC/symmetric-tree	**Easy**	Mirror DFS	[ ]
TREE-05	Subtree of Another Tree	LC/subtree-of-another-tree [B75]	**Easy**	Same-tree check at every node	[ ]
TREE-06	Path Sum	LC/path-sum	**Easy**	Root-to-leaf DFS with remaining sum	[ ]
TREE-07	Diameter of Binary Tree	LC/diameter-of-binary-tree	**Easy**	DFS returns height while updating a global answer	[ ]
TREE-08	Balanced Binary Tree	LC/balanced-binary-tree	**Easy**	Bottom-up height with early exit	[ ]
TREE-09	Binary Tree Inorder Traversal	LC/binary-tree-inorder-traversal	**Easy**	Iterative stack traversal (then Morris)	[ ]
TREE-10	Binary Tree Level Order Traversal	LC/binary-tree-level-order-traversal [B75]	**Medium**	BFS with level-size loop	[ ]
TREE-11	Binary Tree Right Side View	LC/binary-tree-right-side-view	**Medium**	BFS last-of-level / DFS by depth	[ ]
TREE-12	Binary Tree Zigzag Level Order Traversal	LC/binary-tree-zigzag-level-order-traversal	**Medium**	BFS with direction flag	[ ]
TREE-13	Count Good Nodes in Binary Tree	LC/count-good-nodes-in-binary-tree	**Medium**	DFS carrying the path maximum	[ ]
TREE-14	Lowest Common Ancestor of a Binary Tree	LC/lowest-common-ancestor-of-a-binary-tree	**Medium**	Post-order: return found-node signals	[ ]
TREE-15	Lowest Common Ancestor of a BST	LC/lowest-common-ancestor-of-a-binary-search-tree [B75]	**Medium**	Use BST ordering to walk down	[ ]
TREE-16	Validate Binary Search Tree	LC/validate-binary-search-tree [B75]	**Medium**	DFS with (low, high) bounds	[ ]
TREE-17	Kth Smallest Element in a BST	LC/kth-smallest-element-in-a-bst [B75]	**Medium**	Inorder traversal with counter	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
TREE-18	Construct Binary Tree from Preorder and Inorder Traversal	LC/construct-binary-tree-from-preorder-and-inorder-traversal [B75]	**Medium**	Root from preorder, split via inorder index map	[ ]
TREE-19	Flatten Binary Tree to Linked List	LC/flatten-binary-tree-to-linked-list	**Medium**	Reverse post-order / Morris-style rewiring	[ ]
TREE-20	Delete Node in a BST	LC/delete-node-in-a-bst	**Medium**	Three deletion cases with inorder successor	[ ]
TREE-21	Binary Search Tree Iterator	LC/binary-search-tree-iterator	**Medium**	Controlled inorder with explicit stack	[ ]
TREE-22	All Nodes Distance K in Binary Tree	LC/all-nodes-distance-k-in-binary-tree	**Medium**	Parent map + BFS from target	[ ]
TREE-23	Recover Binary Search Tree	LC/recover-binary-search-tree	**Medium**	Inorder anomaly detection (two swapped nodes)	[ ]
TREE-24	Binary Tree Maximum Path Sum	LC/binary-tree-maximum-path-sum [B75]	**Hard**	DFS returns best downward path, global tracks best bent path	[ ]
TREE-25	Serialize and Deserialize Binary Tree	LC/serialize-and-deserialize-binary-tree [B75]	**Hard**	Preorder with null markers / BFS encoding	[ ]
TREE-26	Vertical Order Traversal of a Binary Tree	LC/vertical-order-traversal-of-a-binary-tree	**Hard**	Coordinates (col, row) + sorting	[ ]

Heap / Priority Queue

Learn first: Min/max-heap, top-K, two-heap median, lazy deletion. (Learn before Dijkstra.)

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
HEP-01	Kth Largest Element in a Stream	LC/kth-largest-element-in-a-stream	**Easy**	Min-heap of size k	[ ]
HEP-02	Last Stone Weight	LC/last-stone-weight	**Easy**	Max-heap simulation	[ ]
HEP-03	K Closest Points to Origin	LC/k-closest-points-to-origin	**Medium**	Size-k heap / quickselect	[ ]
HEP-04	Kth Largest Element in an Array	LC/kth-largest-element-in-an-array	**Medium**	Heap or quickselect (average O(n))	[ ]
HEP-05	Top K Frequent Words	LC/top-k-frequent-words	**Medium**	Heap with custom comparator	[ ]
HEP-06	Task Scheduler	LC/task-scheduler	**Medium**	Greedy by max frequency (heap or formula)	[ ]
HEP-07	Reorganize String	LC/reorganize-string	**Medium**	Greedy with max-heap	[ ]
HEP-08	Design Twitter	LC/design-twitter	**Medium**	Merge k feeds via heap	[ ]
HEP-09	Find Median from Data Stream	LC/find-median-from-data-stream [B75]	**Hard**	Two heaps (max-heap low, min-heap high)	[ ]
HEP-10	Sliding Window Median	LC/sliding-window-median	**Hard**	Two heaps with lazy deletion / ordered structure	[ ]
HEP-11	IPO	LC/ipo	**Hard**	Greedy + two heaps	[ ]
HEP-12	Smallest Range Covering Elements from K Lists	LC/smallest-range-covering-elements-from-k-lists	**Hard**	Min-heap over k pointers, track current max	[ ]

Tries

Learn first: Prefix tree node design (children map/array + end flag).

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
TRI-01	Implement Trie (Prefix Tree)	LC/implement-trie-prefix-tree [B75]	**Medium**	Insert / search / startsWith	[ ]
TRI-02	Design Add and Search Words Data Structure	LC/design-add-and-search-words-data-structure [B75]	**Medium**	Trie + DFS for '.' wildcard	[ ]
TRI-03	Replace Words	LC/replace-words	**Medium**	Shortest-prefix lookup in trie	[ ]
TRI-04	Longest Word in Dictionary	LC/longest-word-in-dictionary	**Medium**	Trie / sort + set of buildable prefixes	[ ]
TRI-05	Maximum XOR of Two Numbers in an Array	LC/maximum-xor-of-two-numbers-in-an-array	**Medium**	Binary trie, greedy opposite-bit walk	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
TRI-06	Word Search II	LC/word-search-ii [B75]	**Hard**	Trie-guided grid backtracking with pruning	[ ]

Graphs (BFS/DFS, Topo Sort, DSU, Shortest Paths, MST)

Learn first: Adjacency list, visited sets, multi-source BFS, indegree (Kahn), DSU with path compression, Dijkstra.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
GRA-01	Number of Islands	LC/number-of-islands [B75]	**Medium**	Grid DFS/BFS connected components	[ ]
GRA-02	Max Area of Island	LC/max-area-of-island	**Medium**	Grid DFS with area count	[ ]
GRA-03	Flood Fill	LC/flood-fill	**Easy**	Grid DFS/BFS (revisit as a warm-up)	[ ]
GRA-04	Clone Graph	LC/clone-graph [B75]	**Medium**	DFS/BFS with old-to-new hash map	[ ]
GRA-05	Rotting Oranges	LC/rotting-oranges	**Medium**	Multi-source BFS by level	[ ]
GRA-06	01 Matrix	LC/01-matrix	**Medium**	Multi-source BFS from all zeros	[ ]
GRA-07	Surrounded Regions	LC/surrounded-regions	**Medium**	Border DFS marking safe cells	[ ]
GRA-08	Pacific Atlantic Water Flow	LC/pacific-atlantic-water-flow [B75]	**Medium**	Reverse multi-source DFS from both oceans	[ ]
GRA-09	Shortest Path in Binary Matrix	LC/shortest-path-in-binary-matrix	**Medium**	BFS with 8 directions	[ ]
GRA-10	Number of Provinces	LC/number-of-provinces	**Medium**	Connected components (DFS / DSU)	[ ]
GRA-11	Is Graph Bipartite?	LC/is-graph-bipartite	**Medium**	BFS/DFS 2-coloring	[ ]
GRA-12	Course Schedule	LC/course-schedule [B75]	**Medium**	Cycle detection / Kahn's topological sort	[ ]
GRA-13	Course Schedule II	LC/course-schedule-ii	**Medium**	Topological order output	[ ]
GRA-14	Find Eventual Safe States	LC/find-eventual-safe-states	**Medium**	Reverse graph topo sort / cycle-node marking	[ ]
GRA-15	Graph Valid Tree	LC/graph-valid-tree (Premium) [B75]	**Medium**	n-1 edges + connected (DSU) (Premium)	[ ]
GRA-16	Number of Connected Components in an Undirected Graph	LC/number-of-connected-components-in-an-undirected-graph (Premium) [B75]	**Medium**	DSU / DFS (Premium)	[ ]
GRA-17	Redundant Connection	LC/redundant-connection	**Medium**	DSU: the edge that closes a cycle	[ ]
GRA-18	Accounts Merge	LC/accounts-merge	**Medium**	DSU over emails	[ ]
GRA-19	Word Ladder	LC/word-ladder	**Hard**	BFS over implicit graph (wildcard buckets)	[ ]
GRA-20	Alien Dictionary	LC/alien-dictionary (Premium) [B75]	**Hard**	Build order graph from adjacent words + topo sort (Premium)	[ ]
GRA-21	Network Delay Time	LC/network-delay-time	**Medium**	Dijkstra with min-heap	[ ]
GRA-22	Cheapest Flights Within K Stops	LC/cheapest-flights-within-k-stops	**Medium**	Bellman-Ford (k rounds) / BFS with state	[ ]
GRA-23	Path With Minimum Effort	LC/path-with-minimum-effort	**Medium**	Dijkstra on grid (minimize max edge) / BS + BFS	[ ]
GRA-24	Find the City With the Smallest Number of Neighbors at a Threshold Distance	LC/find-the-city-with-the-smallest-number-of-neighbors-at-a-threshold-distance	**Medium**	Floyd-Warshall all-pairs	[ ]
GRA-25	Min Cost to Connect All Points	LC/min-cost-to-connect-all-points	**Medium**	MST: Prim (heap) / Kruskal (DSU)	[ ]
GRA-26	Swim in Rising Water	LC/swim-in-rising-water	**Hard**	Dijkstra / DSU by time	[ ]
GRA-27	Reconstruct Itinerary	LC/reconstruct-itinerary	**Hard**	Hierholzer's Eulerian path	[ ]
GRA-28	Making A Large Island	LC/making-a-large-island	**Hard**	Label components + try flipping each 0	[ ]
GRA-29	Critical Connections in a Network	LC/critical-connections-in-a-network	**Hard**	Tarjan's bridges (low-link values)	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
GRA-30	Strongly Connected Components (Kosaraju)	GfG: Strongly Connected Components (Kosaraju)	**Hard**	Two-pass DFS with graph transpose (Striver A2Z)	[ ]

Greedy & Intervals

Learn first: Sort by the right key, prove the exchange argument, sweep.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
GRD-01	Assign Cookies	LC/assign-cookies	**Easy**	Sort + two pointers	[ ]
GRD-02	Lemonade Change	LC/lemonade-change	**Easy**	Greedy with change priority	[ ]
GRD-03	Jump Game	LC/jump-game [B75]	**Medium**	Track farthest reachable index	[ ]
GRD-04	Jump Game II	LC/jump-game-ii	**Medium**	BFS-by-levels greedy	[ ]
GRD-05	Gas Station	LC/gas-station	**Medium**	Running surplus, reset start on deficit	[ ]
GRD-06	Partition Labels	LC/partition-labels	**Medium**	Last-occurrence map + sweep	[ ]
GRD-07	Hand of Straights	LC/hand-of-straights	**Medium**	Greedy grouping with ordered counts	[ ]
GRD-08	Merge Triplets to Form Target Triplet	LC/merge-triplets-to-form-target-triplet	**Medium**	Filter triplets exceeding target	[ ]
GRD-09	Valid Parenthesis String	LC/valid-parenthesis-string	**Medium**	Track min/max open-count range	[ ]
GRD-10	Merge Intervals	LC/merge-intervals [B75]	**Medium**	Sort by start, sweep	[ ]
GRD-11	Insert Interval	LC/insert-interval [B75]	**Medium**	Three-phase linear scan	[ ]
GRD-12	Non-overlapping Intervals	LC/non-overlapping-intervals [B75]	**Medium**	Sort by end (activity selection)	[ ]
GRD-13	Minimum Number of Arrows to Burst Balloons	LC/minimum-number-of-arrows-to-burst-balloons	**Medium**	Sort by end, count groups	[ ]
GRD-14	Meeting Rooms	LC/meeting-rooms (Premium) [B75]	**Easy**	Sort and check overlap (Premium)	[ ]
GRD-15	Meeting Rooms II	LC/meeting-rooms-ii (Premium) [B75]	**Medium**	Min-heap of end times / sweep line (Premium)	[ ]
GRD-16	Candy	LC/candy	**Hard**	Two-pass greedy (left then right)	[ ]
GRD-17	Minimum Interval to Include Each Query	LC/minimum-interval-to-include-each-query	**Hard**	Offline sort + min-heap by size	[ ]
GRD-18	N Meetings in One Room / Job Sequencing	GfG: N meetings in one room / Job Sequencing	**Medium**	Activity selection; deadline scheduling (Striver A2Z)	[ ]

Dynamic Programming - 1D & Linear

Learn first: State definition, transition, base case; memoization -> tabulation -> space optimization.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
DP-01	Climbing Stairs	LC/climbing-stairs [B75]	**Easy**	dp[i] = dp[i-1] + dp[i-2]	[ ]
DP-02	Min Cost Climbing Stairs	LC/min-cost-climbing-stairs	**Easy**	Linear DP with min	[ ]
DP-03	House Robber	LC/house-robber [B75]	**Medium**	Take / skip recurrence	[ ]
DP-04	House Robber II	LC/house-robber-ii [B75]	**Medium**	Circular: run twice on two ranges	[ ]
DP-05	Decode Ways	LC/decode-ways [B75]	**Medium**	Counting ways with validity checks	[ ]
DP-06	Word Break	LC/word-break [B75]	**Medium**	Prefix DP with dictionary set	[ ]
DP-07	Coin Change	LC/coin-change [B75]	**Medium**	Unbounded knapsack (min coins)	[ ]
DP-08	Coin Change II	LC/coin-change-ii	**Medium**	Unbounded knapsack (count combinations)	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
DP-09	Longest Increasing Subsequence	LC/longest-increasing-subsequence [B75]	**Medium**	O(n^2) DP -> O(n log n) with patience sorting	[ ]
DP-10	Russian Doll Envelopes	LC/russian-doll-envelopes	**Hard**	Sort + LIS on the second dimension	[ ]
DP-11	Partition Array for Maximum Sum	LC/partition-array-for-maximum-sum	**Medium**	Partition DP with window size k	[ ]

Dynamic Programming - Grid, Knapsack & Subsets

Learn first: 2D states; 0/1 vs unbounded knapsack loops; DP on paths.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
DP-12	Unique Paths	LC/unique-paths [B75]	**Medium**	Grid DP (also nCr)	[ ]
DP-13	Minimum Path Sum	LC/minimum-path-sum	**Medium**	Grid DP with min	[ ]
DP-14	Triangle	LC/triangle	**Medium**	Bottom-up DP with O(n) space	[ ]
DP-15	Maximal Square	LC/maximal-square	**Medium**	dp = 1 + min(top, left, diag)	[ ]
DP-16	Partition Equal Subset Sum	LC/partition-equal-subset-sum	**Medium**	0/1 knapsack (subset-sum boolean)	[ ]
DP-17	Target Sum	LC/target-sum	**Medium**	Subset-sum count via sum transformation	[ ]
DP-18	Longest Increasing Path in a Matrix	LC/longest-increasing-path-in-a-matrix	**Hard**	DFS + memoization on DAG	[ ]
DP-19	Cherry Pickup	LC/cherry-pickup	**Hard**	Two-walker simultaneous DP	[ ]
DP-20	Super Egg Drop	LC/super-egg-drop	**Hard**	DP with binary search / moves-based state	[ ]

Dynamic Programming - Strings & Stock Patterns

Learn first: LCS family (two-string 2D table); state machine DP for stock problems.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
DP-21	Longest Common Subsequence	LC/longest-common-subsequence [B75]	**Medium**	2D DP on two strings	[ ]
DP-22	Longest Palindromic Subsequence	LC/longest-palindromic-subsequence	**Medium**	LCS of s and reverse(s) / interval DP	[ ]
DP-23	Edit Distance	LC/edit-distance	**Medium**	Insert / delete / replace transitions	[ ]
DP-24	Interleaving String	LC/interleaving-string	**Medium**	2D boolean DP over two prefixes	[ ]
DP-25	Distinct Subsequences	LC/distinct-subsequences	**Hard**	Count matches: take / skip on two strings	[ ]
DP-26	Shortest Common Supersequence	LC/shortest-common-supersequence	**Hard**	LCS table + reconstruction	[ ]
DP-27	Regular Expression Matching	LC/regular-expression-matching	**Hard**	2D DP with '*' transitions	[ ]
DP-28	Wildcard Matching	LC/wildcard-matching	**Hard**	2D DP with '*' and '?'	[ ]
DP-29	Best Time to Buy and Sell Stock with Cooldown	LC/best-time-to-buy-and-sell-stock-with-cooldown	**Medium**	State machine: hold / sold / rest	[ ]
DP-30	Best Time to Buy and Sell Stock III	LC/best-time-to-buy-and-sell-stock-iii	**Hard**	State machine with at most 2 transactions	[ ]

Dynamic Programming - Interval, Tree & Bitmask

Learn first: Interval DP (split point k), tree DP (return pairs), bitmask DP (n <= 20).

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
DP-31	Burst Balloons	LC/burst-balloons	**Hard**	Interval DP: choose the LAST balloon to burst	[ ]
DP-32	House Robber III	LC/house-robber-iii	**Medium**	Tree DP returning (rob, skip)	[ ]

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
DP-33	Shortest Path Visiting All Nodes	LC/shortest-path-visiting-all-nodes	**Hard**	BFS over (node, visited-mask) states	[ ]
DP-34	Partition to K Equal Sum Subsets	LC/partition-to-k-equal-sum-subsets	**Medium**	Bitmask DP alternative to backtracking	[ ]

Math & Number Theory

Learn first: Sieve, GCD, fast exponentiation, modular arithmetic.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
MTH-01	Roman to Integer	LC/roman-to-integer	**Easy**	Left-to-right with subtractive rule	[ ]
MTH-02	Happy Number	LC/happy-number	**Easy**	Cycle detection on digit-square sums	[ ]
MTH-03	Plus One	LC/plus-one	**Easy**	Carry propagation	[ ]
MTH-04	Count Primes	LC/count-primes	**Medium**	Sieve of Eratosthenes	[ ]
MTH-05	Factorial Trailing Zeroes	LC/factorial-trailing-zeroes	**Medium**	Count factors of 5	[ ]
MTH-06	Multiply Strings	LC/multiply-strings	**Medium**	Grade-school multiplication with digit arrays	[ ]
MTH-07	Fraction to Recurring Decimal	LC/fraction-to-recurring-decimal	**Medium**	Long division with remainder map	[ ]

Advanced Structures (Segment Tree, Fenwick, Merge-Sort Counting)

Learn first: Only after Tiers 1-4 feel solid. Needed for Codeforces/AtCoder, rarely for campus interviews.

ID	Problem Name	LeetCode Link / Platform	Difficulty	Key Pattern	Done
ADV-01	Range Sum Query - Mutable	LC/range-sum-query-mutable	**Medium**	Fenwick tree / segment tree point update + range query	[ ]
ADV-02	Count of Smaller Numbers After Self	LC/count-of-smaller-numbers-after-self	**Hard**	Fenwick tree with coordinate compression / merge sort	[ ]
ADV-03	Reverse Pairs (revisit)	LC/reverse-pairs	**Hard**	Merge-sort counting or Fenwick tree	[ ]
ADV-04	Range Sum Query 2D - Immutable	LC/range-sum-query-2d-immutable	**Medium**	2D prefix sums	[ ]
ADV-05	The Skyline Problem	LC/the-skyline-problem	**Hard**	Sweep line + max-heap	[ ]

PART 3: High-Yield Company-Wise Filters

Read this as directional, not official. Companies do not publish question banks; these mappings reflect widely reported interview themes. For live frequency data, filter by company on LeetCode (Premium) and cross-check with recent interview experiences on LeetCode Discuss and GeeksforGeeks.

Company (FAANG+M)	Most Frequent Patterns	Top 3 Specific Must-Solve Problems
Google	Graph traversal (BFS/DFS), DP, binary search on answer, string/array manipulation, intervals	1. Word Ladder 2. Longest Increasing Path in a Matrix 3. Median of Two Sorted Arrays
Amazon	Grid BFS/DFS, heap / top-K, hash maps, linked lists, sliding window, LRU-style design	1. Number of Islands 2. K Closest Points to Origin 3. LRU Cache
Meta (Facebook)	Arrays/strings, prefix sums + hash map, binary trees, graphs, binary search, stack-based parsing	1. Subarray Sum Equals K 2. Lowest Common Ancestor of a Binary Tree 3. Minimum Remove to Make Valid Parentheses
Apple	Arrays/strings, hashing, trees, linked lists, backtracking	1. Longest Substring Without Repeating Characters 2. Group Anagrams 3. Word Search
Netflix	Hash-map design problems, top-K/heap, intervals, time-keyed storage, practical data-structure design	1. Time Based Key-Value Store 2. Top K Frequent Elements 3. Insert Delete GetRandom O(1)
Microsoft	Linked lists, trees, arrays/matrix, strings, basic-to-medium DP, design	1. Copy List with Random Pointer 2. Serialize and Deserialize Binary Tree 3. Spiral Matrix

Universal high-yield core (asked across all six)

Two Sum, Merge Intervals, Number of Islands, LRU Cache, Word Break, Course Schedule, Lowest Common Ancestor of a Binary Tree, Top K Frequent Elements, Merge k Sorted Lists, Longest Substring Without Repeating Characters. If time is short, master these ten patterns before touching company-specific lists.

Pattern cheat sheet: problem signal to technique

If the problem says / shows...	Reach for...
Sorted input, or a monotonic yes/no answer space	Binary search (on index or on the answer)
Contiguous subarray/substring with a constraint	Sliding window (all-positive) or prefix sum + hash map (negatives allowed)
Pair/triplet in a sorted array; in-place rearrangement	Two pointers
Next greater/smaller element; nearest larger to the left/right	Monotonic stack
Max/min in each window	Monotonic deque
Top K, K-th, running median, merging K streams	Heap (two heaps for median)
Shortest path, unweighted grid or graph	BFS (multi-source if several starts)
Shortest path, non-negative weights	Dijkstra
Ordering with prerequisites/dependencies	Topological sort (Kahn / DFS)
Grouping, 'are these connected?', merging sets	DSU (Union-Find)

If the problem says / shows...	Reach for...
Generate all / n <= 15-20	Backtracking or bitmask
Count ways, min/max with choices, overlapping subproblems	DP (state, transition, base case)
Prefix/dictionary queries on many strings	Trie
Sorting by one key unlocks a local-choice proof	Greedy (exchange argument)
n <= 1e5 -> O(n log n); n <= 1e3 -> O(n^2); n <= 20 -> exponential	Read constraints first, choose the complexity budget

PART 4: Daily Strategy & Tracking Template

4.1 Time boxes per problem

Difficulty	Think before coding	Hard stop	After the cap
Easy	5 min: restate, examples, brute force	20 min total	Read the pattern tag; retry for 10 min
Medium	10 min: brute force, then find the bottleneck	35 min total	Hint ladder below; re-implement from scratch after
Hard	15 min: constraints, related Medium you already know	50 min total	Read approach (not code); solve next day

4.2 Hint ladder (climb one rung at a time, never jump to the code)

Rung	Action
1	Re-read the constraints and write the brute force. Name its bottleneck.
2	Look only at the topic/pattern tag (e.g. 'monotonic stack'), then try again for 10 minutes.
3	Read one hint or the first line of the discussion approach.
4	Read the full approach in words, close it, code it yourself.
5	Read the code only if rung 4 failed. Re-solve from a blank editor within 24 hours. Mark the problem **Red**.

4.3 Spaced repetition

Rate every solve: **Green** = solved cleanly, no hint. **Yellow** = solved with a hint or over the time cap. **Red** = needed the solution. Schedule revisits by colour.

Rating	R1	R2	R3	R4	R5
Red	Day +1	Day +3	Day +7	Day +14	Day +30
Yellow	-	Day +3	Day +7	Day +21	-
Green	-	-	Day +7	-	Day +30 (then retire)

Rule: a revision counts only if you solve it from a blank editor. If you fail a revision, the problem drops back to Red and the ladder restarts.

4.4 Daily session template (about 2.5 hours)

Block	What to do
10 min	Revision queue: redo the day's due Red/Yellow problems (max 2)
90 min	New problems: 2 Medium, or 1 Medium + 1 Easy, from the current Part 2 topic
25 min	Upsolve: study one alternative solution for each problem, note the pattern
10 min	Log everything in the tracker below (rating, mistake, revision dates)
Minimum day	One problem plus its log entry. Never zero. Streaks come from a floor, not from heroic days.

4.5 Weekly cadence

Day	Focus
Mon-Thu	New-topic days: Part 2, in order
Fri	Revision day: clear the whole Red/Yellow queue, no new problems
Sat	Contest day: LeetCode Weekly/Biweekly, Codeforces Div 3/4, or an AtCoder ABC (virtual is fine). Upsolve the problem you missed
Sun	Rest half day + 20 min weekly review: counts per topic, top 3 recurring mistakes, plan for next week

4.6 Twelve-week roadmap

Week	Focus	Target
1	Part 1 (all five drills)	40 problems
2	Arrays & Hashing, Strings	About 25
3	Binary Search, Linked List (start weekly contests)	About 25
4	Stacks & Monotonic Structures, Recursion & Backtracking	About 25
5-6	Trees & BSTs, Heaps, Tries	About 50
7-8	Graphs (BFS/DFS, topo sort, DSU, Dijkstra, MST)	About 30
9	Greedy & Intervals	About 15
10-11	Dynamic Programming (all four tables)	About 45
12	Math, Advanced Structures, mock interviews, full revision of all Reds	About 12 + mocks

4.7 Progress dashboard

Topic (Part 2)	Problems	Solved	Reds still open	Completed on
Arrays & Hashing	20			
Strings (Advanced Windows, Palindromes, KMP)	14			
Binary Search	15			
Linked List	17			
Stacks, Queues & Monotonic Structures	17			
Recursion & Backtracking	11			
Trees & Binary Search Trees	26			
Heap / Priority Queue	12			
Tries	6			
Graphs (BFS/DFS, Topo Sort, DSU, Shortest Paths, MST)	30			
Greedy & Intervals	18			
Dynamic Programming - 1D & Linear	11			
Dynamic Programming - Grid, Knapsack & Subsets	9			
Dynamic Programming - Strings & Stock Patterns	10			
Dynamic Programming - Interval, Tree & Bitmask	4			
Math & Number Theory	7			
Advanced Structures (Segment Tree, Fenwick, Merge-Sort Counting)	5			
Total	**232**			

4.8 Daily log (photocopy or re-create in a spreadsheet)

Date	ID	Problem	Min	Hint?	G/Y/R	Pattern / mistake to avoid	R1	R2	R3	R4	R5

Date	ID	Problem	Min	Hint?	G/Y/R	Pattern / mistake to avoid	R1	R2	R3	R4	R5

Appendix: Codeforces / AtCoder / CSES bridge

The master sheet covers interview patterns. Contest sites reward speed and a wider toolbox. Use this ladder in parallel from week 3.

Step	Resource / target	Where	Why this order
1	CSES Problem Set: Introductory, then Sorting and Searching	cses.fi/problemset	Clean, statement-only problems that drill the Tier 0-3 toolbox
2	AtCoder Beginner Contests, problems A-D	atcoder.jp/contests	Well-tested problems, no hacks; A-D maps closely to Medium-level interview patterns
3	Codeforces Div 3 / Div 4 (virtual), rating 800-1200	codeforces.com/problemset	Adds implementation speed and greedy/constructive thinking
4	CSES: Dynamic Programming, Graph Algorithms	cses.fi/problemset	Same topics as Part 2 DP and Graphs, at contest depth
5	CP-Algorithms: segment tree, Fenwick tree, LCA, string hashing	cp-algorithms.com	Reference for the Advanced table; learn a tool only when a contest problem needs it
6	AtCoder ABC problem E and Codeforces Div 2 (B-C)	atcoder.jp / codeforces.com	Next tier once A-D feel routine

Language note: C++ is the safer contest choice for time limits; Python is fine for LeetCode and most ABC A-D problems. Pick one and do not switch mid-plan.